



DO NOT LEAK SECRETS

USABILITY TESTING ENGINEER TOOLING

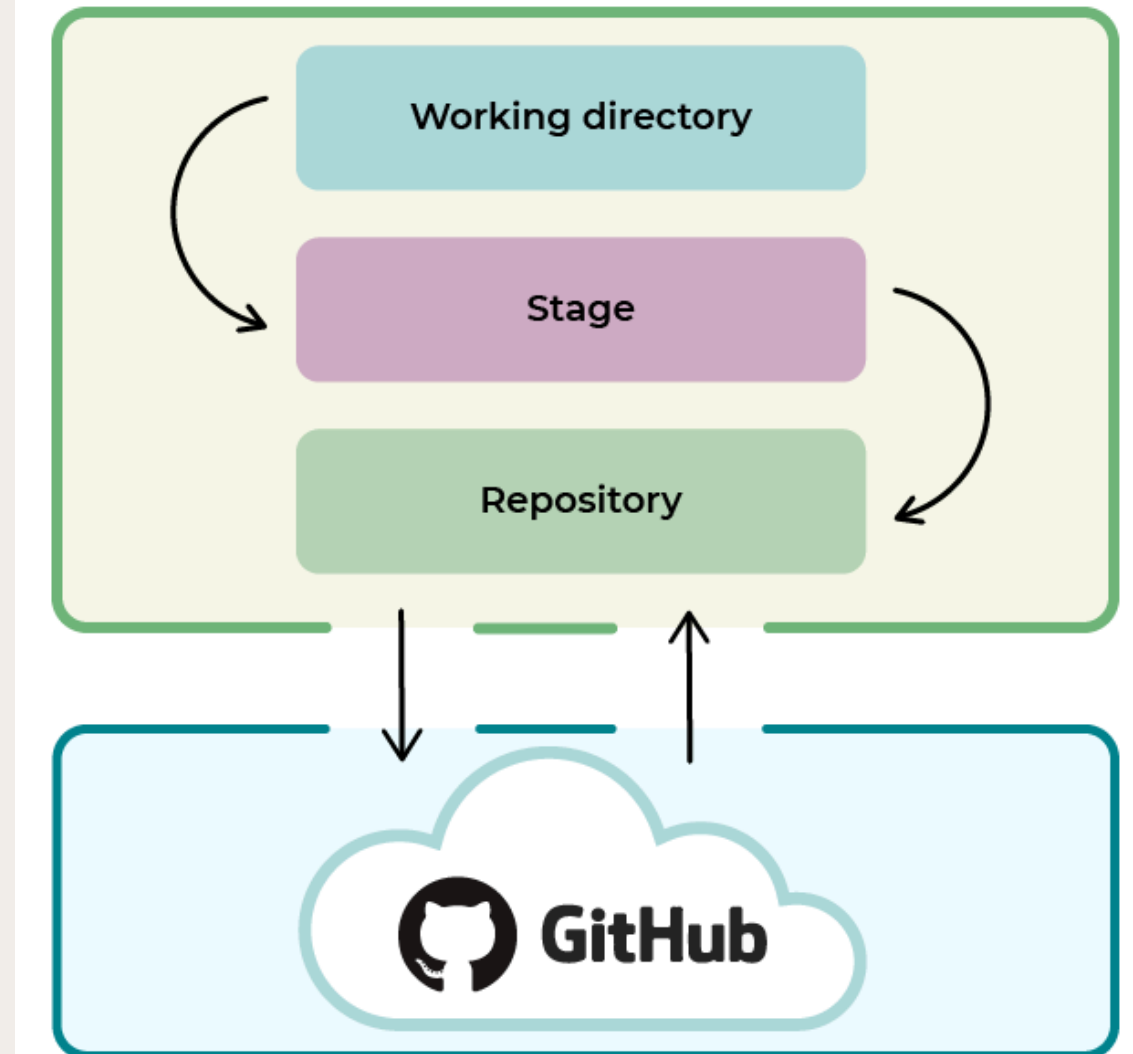
TABLE OF CONTENTS

- 3 - The product
- 4 - Pain points
- 5 - Advocating for research
- 6 - Research questions
- 7 - Research plan
- 8 - Scenario for usability test
- 9, 10 -Screens from prototype
- 11 - Research findings & impact
- 12 - Conclusion
- 13 - Credits

THE PRODUCT

Github repository

- Like Google Docs, but for code
- Developer pushes their code to the internet
 - Like “upload” or “sync”
- Even after pushing new code, previous versions are visible
 - Like with Google Docs’ version history, can see who changed what, and when



PAIN POINT

 Problem	Secrets (like API keys, passwords) are sometimes unfortunately pushed to a repository.
 Partial solution	Scan repositories looking for secrets, alert developers if/when one is found.
 Better solution	• But it would be better if secrets were never pushed! • Build a guardrail that keeps secrets from being pushed to a repository.



ADVOCATING FOR RESEARCH

- PM was ready to move forward with launch
 - Argued “push protection” is same thing as “secret scanning”
- I advocated we do some testing
 - No prior research about secret scanning
 - Push protection is more complicated:
 - Did we actually build *guardrails* - not a speed bump, or high security prison?
 - Is it usable for different user groups?



RESEARCH QUESTIONS

1. Do users understand what push protection is, and why it matters?
2. Can users successfully resolve a blocked push?
3. How are experiences different for different users?

RESEARCH PLAN

- Method: Usability test
- Participants
 - 5 junior developers (6 months to 2 years)
 - 5 senior developers (5-15 years)
- Difficulty: Finding qualified participants?
- Solution: Resondent.io, LinkedIn.com, GitHub profiles

USABILITY TEST SCENARIO

- Difficulty:
 - Software & hardware limitations
- Solution:
 - Brainstorming with PM to identify options
 - Finally: founding an organization on GitHub, adjusting settings, creating a repository.
- Difficulty:
 - How do we devise a realistic scenario?
 - Developers usually write & push their own code
 - Most employers have policies on secrets
- Solution:
 - Creating separate scenarios for junior, senior developers

SUCCESSFUL `GIT PUSH`

```
justi@JM-DESKTOP MINGW64 ~/OneDrive/Documents/dev_code/new-test-repo (main)
```

```
$ git push origin main
```

```
Enumerating objects: 4, done.
```

```
Counting objects: 100% (4/4), done.
```

```
Delta compression using up to 20 threads
```

```
Compressing objects: 100% (2/2), done.
```

```
Writing objects: 100% (3/3), 351 bytes | 351.00 KiB/s, done.
```

```
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
```

```
To https://github.com/jumuir/new-test-repo.git
```

```
fa232f0..6bfdc5e  main -> main
```

```
justi@JM-DESKTOP MINGW64 ~/OneDrive/Documents/dev_code/new-test-repo (main)
```

```
$ |
```

BLOCKED BY PUSH PROTECTION

```
→ ~/my_project git:(branch_name) git push
remote: error GH009: Secrets detected! This push failed.
remote:
remote:          GITHUB PUSH PROTECTION
remote: _____
remote:  Resolve the following secrets before pushing again.
remote:
remote:  (?) Learn how to rewrite your local commit history
remote:  https://git-scm.com/book/en/v2/Git-Tools-Rewriting-History
remote:
remote:
remote: — GitHub Personal Access Token —————
remote:  locations:
remote:    - commit: c47ff8afc1ce530798ce62e064c28fe26c33c99b
remote:      path: src/config/credentials.js:12
remote:
remote:  (?) To push, remove secret from commit(s) or follow this
URL to allow the secret.
```

FINDING	IMPACT
Most participants skimmed error messages, some did not scroll to the top of error messages.	Adjust our user personas - we were surprised that both senior and junior developers made this mistake!
Most senior developers found it intuitive to resolve the issue.	Validated our decision to speak with different kinds of users.
But some users struggled to resolve the issue • Unaware that secret was in the history • Knew that, but unsure how to resolve	New design • Improved content comprehension by 40% • Increased task completion by 25%
All participants easily understood the value of this feature.	Validated decision to move forward. After making improvements, rolled out feature to all repositories!

CONCLUSION

- Stakeholder management
 - Persuaded stakeholders why research was necessary
- Research prioritization & planning
 - Scoped the project, given the time frame
 - Identified feasible ways of conducting usability tests
- Creating business impact
 - Recommend content adjustments as low-lift solutions
 - Rolled out the feature on-time



CREDITS

Title slide Hayyan Creative Studio

Deck template Saga Design Studio